VeriVote Kenya

8-Week Development Roadmap

(Modified: Web-First Approach)

January 26 – March 22, 2026

Executive Summary

This roadmap outlines an 8-week development plan for VeriVote Kenya — a sovereign hybrid electronic voting system built for Kenya. The system is designed for full national independence: all AI inference, encryption, and data processing run on-premise with zero dependency on foreign APIs or cloud platforms. Week 6 delivers a locally-deployed AI fraud detection layer (Isolation Forest + Llama 3.2 via Ollama) that operates fully air-gapped, directly addressing NIRU judging criteria A2 (National Self-Reliance) and Final Stage Criterion 2 (National Sovereignty).

Key Modifications from Original Plan

- Web application built first with reusable REST API endpoints for future mobile app
- Simplified verification: Digital receipt instead of QR code mobile scanning
- Centralized vote printing system for physical backup and audit trail
- All other features (blockchain, AI, encryption, distress PIN) maintained

Development Strategy

- Build incrementally: Each week produces working, testable code
- API-first design: All features exposed via REST endpoints
- Prioritize demonstrability: Focus on features that showcase the concept
- Use simulations: Where full implementation is complex, use mock data
- Leave buffer: Week 8 dedicated to integration, testing, and demo preparation

MVP Feature Scope

IN SCOPE (MVP)	SIMPLIFIED	SIMULATED	FUTURE
Basic SBT system	Homomorphic encryption (basic)	AI fraud detection (demo data)	Full Hyperledger
Voter registration	ZKP (simplified)	Mixnet (visualization)	Production ZKP
Vote casting UI	Distress PIN	Multi-node consensus	Mobile app
Digital receipt	Public portal		Advanced AI
Web verification			
Vote printing			
Basic blockchain			

Weekly Milestones

Week 1: Foundation & Infrastructure Setup (Jan 26 – Feb 1, 2026)

Goal: Set up development environment and core infrastructure

Day 1-2: Project Setup (Jan 26-27)

- Initialize monorepo structure (backend, frontend, smart-contracts)
- Set up Git repository with branching strategy
- Configure TypeScript + Node.js backend with Express
- Set up PostgreSQL database with initial schema
- Configure Redis for caching
- Set up Docker containers for local development
- Create basic CI/CD pipeline (GitHub Actions)

Day 3-4: Database Schema & Core Models (Jan 28-29)

- Design and implement Voter table (id, national_id, sbt_address, created_at)
- Design and implement Vote table (encrypted_vote_hash, timestamp, serial_number)
- Design and implement PollingStation table
- Design and implement PrintQueue table for centralized printing
- Create database migrations
- Write seed data for testing (100 test voters, 10 polling stations)
- Create TypeORM/Prisma models and repositories

Day 5-7: Basic Blockchain Setup (Jan 30 – Feb 1)

- Set up local Ethereum testnet (Hardhat/Ganache) OR Polygon Mumbai testnet
- Write basic Soul-Bound Token (SBT) smart contract in Solidity
- Implement mint() function (non-transferable tokens)
- Write Vote Recording smart contract (store vote hashes)
- Deploy contracts to testnet

- Create backend service to interact with contracts (ethers.js/web3.js)
- Write unit tests for smart contracts

Week 1 Deliverables

- Working development environment
- Database with core tables and seed data
- Deployed SBT and Vote Recording smart contracts on testnet
- Basic API endpoints for voter registration

Week 2: Voter Registration & Authentication (Feb 2-8, 2026)

Goal: Complete voter registration system with SBT minting

Day 8-10: Voter Registration Backend (Feb 2-4)

- Create API endpoint: POST /api/voters/register
- Implement national ID validation (mock biometric check)
- Generate Ethereum wallet for new voter
- Mint SBT token and assign to voter's wallet
- Store voter record in PostgreSQL with SBT address
- Implement PIN generation (normal + distress PIN)
- Hash PINs using Argon2 (via libsodium/Node.js crypto)
- Create API endpoint: POST /api/voters/verify-pin

Day 11-12: Authentication Flow (Feb 5-6)

- Implement JWT-based session management
- Create middleware for authentication checks
- Implement rate limiting (prevent brute force attacks)
- Add voter status tracking (registered, voted, used_distress_pin)
- Write API endpoint: GET /api/voters/:id/status

Day 13-14: Admin Dashboard (Basic) (Feb 7-8)

- Create React admin portal (Next.js)
- Build voter registration form
- Display registered voters list with SBT addresses
- Show blockchain confirmation for SBT minting
- Add basic statistics dashboard (total voters, SBTs minted)

Week 2 Deliverables

- Complete voter registration API
- SBT minting working end-to-end
- PIN authentication (normal + distress)
- Basic admin dashboard showing registered voters

Week 3: Vote Casting & Cryptographic Core (Feb 9-15, 2026)

Goal: Build vote casting interface with encryption and receipt generation

Day 15-17: Web-Based Voting Interface (Feb 9-11)

- Create React voting interface (Next.js web application)
- Build PIN entry screen
- Create candidate selection UI (presidential race + sample down-ballot)
- Implement vote review screen
- Add multi-language support (English/Swahili)
- Create accessible UI (large buttons, high contrast)
- Design responsive layout for various screen sizes

Day 18-19: Basic Encryption & Hashing (Feb 12-13)

- Install Microsoft SEAL (or use simpler library for MVP)
- Implement basic homomorphic encryption OR use AES-256 + plan upgrade path
- Create vote serialization format (JSON → encrypted → hash)
- Generate SHA-256 hash of encrypted vote
- Create API endpoint: POST /api/votes/cast

Day 20-21: Receipt Generation & Blockchain Recording (Feb 14-15)

- Generate unique serial number for each vote
- Create digital receipt with: serial number, timestamp, blockchain tx hash
- Build receipt display page showing vote confirmation
- Create API endpoint: GET /api/receipts/:serial
- Write blockchain transaction to record vote hash
- Update smart contract to emit VoteCast event
- Store encrypted vote in PostgreSQL as backup
- Update voter status to 'voted'

Week 3 Deliverables

- Functional web-based voting interface
- Vote encryption working
- Digital receipt generation with serial number
- Votes recorded on blockchain
- End-to-end vote casting flow working

Week 4: Web Verification Portal & Print System (Feb 16-22, 2026)

Goal: Build web-based verification and centralized vote printing for backup

Day 22-24: Web Verification System (Feb 16-18)

- Create web-based receipt verification page
- Build serial number lookup interface
- Implement cryptographic verification (server-side)
- Verify vote hash matches encrypted payload
- Query blockchain API to confirm vote is recorded
- Create API endpoint: GET /api/votes/verify/:serial
- Display verification result page (checkmark/error)
- Show vote details (timestamp, blockchain confirmation)
- Add printable receipt option for voters

Day 25-26: Centralized Vote Printing System (Feb 19-20)

- Design print queue database schema
- Create API endpoint: POST /api/print-queue/add
- Build print job processing service
- Implement secure print format (vote hash, serial, timestamp)
- Create admin interface for print queue management
- Build batch printing functionality
- Add print status tracking (queued, printing, completed)
- Implement print verification/reconciliation

Day 27-28: API Documentation & Mobile-Ready Endpoints (Feb 21-22)

- Document all REST API endpoints with Swagger/OpenAPI
- Ensure all endpoints return consistent JSON responses
- Add CORS configuration for future mobile app
- Create API authentication tokens for mobile clients
- Build API rate limiting per client
- Test all endpoints for mobile compatibility
- Create API versioning strategy (v1)

Week 4 Deliverables

- Working web-based verification portal
- Receipt verification with blockchain query
- Centralized vote printing system operational

- Print queue management interface
- Fully documented REST API ready for mobile consumption

Week 5: Public Portal & Transparency Features (Feb 23 – Mar 1, 2026)

Goal: Build public verification portal and blockchain explorer

Day 29-31: Public Verification Portal (Feb 23-25)

- Create Next.js portal with responsive design
- Build homepage with real-time election statistics
- Create vote lookup page (by serial number)
- Display blockchain confirmation and timestamp
- Build blockchain explorer page
- Show recent blocks with vote counts
- Add turnout statistics (by region, polling station)

Day 32-33: Real-Time Dashboard (Feb 26-27)

- Set up WebSocket connection for real-time updates
- Create live vote counter
- Build polling station status map (Google Maps or simple)
- Add charts for turnout trends (Chart.js/Recharts)
- Display system health indicators (blockchain, API status)
- Add print queue statistics to dashboard

Day 34-35: Distress PIN Handler & Multiple Voting (Feb 28 – Mar 1)

- Implement distress PIN detection in backend
- Create silent flagging system (no visible indication)
- Build notification system (SMS via Africa's Talking API)
- Implement multiple voting logic (supersede previous vote)
- Create admin interface to view distress PIN alerts
- Add time-lock protection (unlimited votes first 12hrs, freeze last 2hrs)

Week 5 Deliverables

- Public verification portal live
- Blockchain explorer functional
- Real-time dashboard with statistics
- Distress PIN system working
- Multiple voting capability implemented

Week 6: Sovereign AI Security Layer & Tallying (Mar 2-8, 2026)

Goal: Implement locally-deployed AI fraud detection with full data sovereignty — no voter data leaves Kenyan infrastructure

Day 36-38: Anomaly Detection & Rule Engine (Mar 2-4)

- Set up Python microservice for AI models (FastAPI) — runs fully on-premise
- Install scikit-learn, numpy, pandas; create Isolation Forest anomaly detector
- Generate synthetic Kenyan election training data (simulate fraud patterns locally)
- Train model on 5 features: voting_velocity, temporal_deviation, geographic_cluster_score, repeat_attempt_rate, distress_correlation
- Create API endpoint: POST /api/ai/analyze-voting-pattern → returns anomaly_score 0–100
- Build deterministic rule engine (fully auditable): 3+ distress votes same station/30min → CRITICAL; velocity > 3× average → HIGH
- Implement alerting: score > 70 triggers alert pipeline; all decisions logged with feature values for audit (D2 Transparency)
- No external API calls at any stage — entire detection pipeline runs on local Kenyan infrastructure (A2 National Self-Reliance)

Day 39-40: Sovereign LLM Integration for Explainability (Mar 5-6)

- Deploy Ollama locally on hackathon GPU (T4) — open-source LLM runtime, zero data egress
- Pull Llama 3.2 8B (4-bit quantised, fits in ~5GB VRAM) — no foreign API dependency at runtime
- Write prompt templates: feed anomaly_score + feature values + rule triggers → Llama → plain-English alert for IEBC officers
- Generate contextual risk assessment: 'Station X shows 3× normal velocity with 2 distress flags since 14:00 — recommend immediate verification'
- Build AI security dashboard in admin portal: flagged patterns list, anomaly score trend chart, per-station risk level, LLM alert feed
- All LLM inference logs stored locally — supervisors can reconstruct every explanation (Stage 2 D2 Auditability)
- Sovereignty note: system operates fully air-gapped; no OpenAI / foreign API keys required in production (Final Stage Criterion 2)

Day 41-42: Decryption Ceremony & Election Results (Mar 7-8)

- Implement ElGamal decryption ceremony: IEBC admin triggers tally — encrypted votes decrypted in batch using private key held by system
- NOTE: ElGamal as implemented is multiplicatively homomorphic (not additively); decryption ceremony approach is correct for this stack — SEAL/Paillier upgrade path documented for future
- Simulate threshold key custody: display ceremony log showing each vote decrypted and counted (auditability demonstration)

- Build results display page: candidate vote counts, turnout percentage, station-by-station breakdown
- Add cryptographic proof of tally: SHA-256 hash of full results JSON published on-chain as tamper-evident seal
- Create downloadable audit report (PDF): tally summary, distress flag count, anomaly alerts fired, AI scores, blockchain TX references
- Include print queue reconciliation: compare printed ballot count vs digital tally — flag discrepancies
- Sovereignty checkpoint: confirm full election cycle completed with zero foreign API dependencies

Week 6 Deliverables

- AI fraud detection system operational
- Real-time anomaly scoring
- OpenAI-powered alert generation
- AI security dashboard
- Basic tallying system with results display

Week 7: Polish, Security, and Simulations (Mar 9-15, 2026)

Goal: Add missing features, build simulations, and harden security

Day 43-44: Mixnet Simulation (Mar 9-10)

- Create visual mixnet demonstration
- Build animated UI showing vote shuffling through nodes
- Implement simulated Chaumian mixing (visualization only)
- Show ZKP verification at each shuffle stage
- Add this to public portal as education tool

Day 45-46: Security Hardening (Mar 11-12)

- Conduct security audit of all API endpoints
- Implement rate limiting on all public endpoints
- Add input validation and sanitization
- Set up HTTPS/TLS for all connections
- Configure CORS policies
- Add helmet.js security headers
- Implement logging and monitoring (Winston/Datadog)
- Set up error tracking (Sentry)
- Secure print queue endpoints

Day 47-49: Demo Data & Scenarios (Mar 13-15)

- Create comprehensive demo dataset (1000 voters, 100 polling stations)
- Generate realistic voting patterns
- Simulate fraud scenarios (for AI detection demo)
- Create coercion scenario (distress PIN usage)
- Build demo script for live presentation
- Record video walkthrough of key features
- Prepare printed vote samples for demo

Week 7 Deliverables

- Mixnet visualization complete
- Security hardening complete
- Comprehensive demo dataset
- Fraud simulation scenarios
- Demo script ready

Week 8: Integration, Testing & Final Presentation (Mar 16-19, 2026)

Goal: Complete end-to-end testing, fix bugs, and prepare final demo

Day 50-52: End-to-End Testing (Mar 16)

- Run complete election simulation (50+ voters)
- Test all user journeys (voter, admin, observer)
- Verify blockchain integrity
- Test web interface on multiple browsers/devices
- Load testing (simulate 1000+ concurrent voters)
- Fix critical bugs
- Test distress PIN flow end-to-end
- Verify AI alerts trigger correctly
- Test print queue under load
- Verify receipt verification flow

Day 53-54: Documentation (Mar 17)

- Write comprehensive README.md
- Create API documentation (Swagger/OpenAPI)
- Document deployment process
- Write technical architecture document
- Create user guide (PDF)
- Record demo videos for each feature
- Document mobile API integration guide for future development

Day 55-56: Final Presentation Prep (Mar 19)

- Create pitch deck (PowerPoint/Google Slides)
- Rehearse live demo
- Prepare backup demo (in case live fails)
- Deploy to cloud (AWS/Heroku/Vercel)
- Set up public demo site
- Test demo on presentation hardware
- Prepare Q&A responses
- Final polish on UI/UX

Week 8 Deliverables

• Fully tested, bug-free system

• Complete documentation

- Live demo site deployed
- Presentation materials ready
- Project ready for demo

Technical Implementation Details

Core Technology Stack

Category	Technologies
Backend	Node.js v20+, TypeScript v5+, Express.js, PostgreSQL v16+, Redis v7+
Frontend	React v18+, Next.js v14+, Tailwind CSS v3.4+, Chart.js v4+
Blockchain	Hardhat, Polygon Mumbai Testnet, Solidity v0.8.20+, ethers.js, OpenZeppelin
Cryptography	Microsoft SEAL (optional), libsodium, Node.js crypto, snarkjs (simplified)
AI/ML	Python v3.11+, FastAPI, scikit-learn v1.4+, OpenAI API (GPT-4o-mini)
Printing	Node-printer, PDF generation (pdfkit), Print queue service
DevOps	Docker, GitHub Actions, AWS/Heroku/Vercel, PM2, Winston, Sentry

Backend API: Node.js 20 + TypeScript + Express.js + Prisma ORM

Database: PostgreSQL 16 (primary) + Redis 7 (OTP cache / rate limiting)

Blockchain: Hardhat local EVM testnet (Polygon RPC for production); ethers.js; Solidity SBT contract

Encryption: ElGamal 2048-bit FFDHE RFC 7919 (vote encryption); Argon2id (PIN hashing); SHA-256 (vote hash)

Identity / KYC: Persona (ID document + liveness, inline iframe); WebAuthn / FIDO2 (fingerprint at polling station)

Frontend: Next.js 15 + React + Tailwind CSS + Recharts + Socket.IO client

Real-Time: Socket.IO (WebSocket + polling fallback) — live vote counter, distress alerts

Notifications: Africa's Talking (SMS, Kenyan provider); Nodemailer SMTP (email)

AI — Detection: Python FastAPI + scikit-learn Isolation Forest + deterministic rule engine (fully local)

AI — Explanation: Ollama + Llama 3.2 8B (locally deployed, 4-bit quantised, GPU T4) — ZERO foreign API dependency

Sovereignty: All runtime services run on Kenyan infrastructure; no voter data transmitted abroad

Repository Structure

verivote-kenya/

```
├── backend/                    # Node.js + TypeScript API
```

```
│   └── src/
│       ├── controllers/        # API route handlers
│       ├── models/             # Database models
│       ├── services/           # Business logic
│       ├── blockchain/         # Smart contract interactions
│       └── print-service/      # Centralized printing
├── frontend/                   # Next.js web portal (voting + public)
│   └── pages/
│       ├── vote/               # Voting interface
│       ├── verify/             # Receipt verification
│       ├── admin/              # Admin dashboard
│       └── public/             # Public statistics
├── smart-contracts/            # Solidity contracts
│   └── contracts/
│       ├── SoulBoundToken.sol
│       └── VoteRecorder.sol
├── ai-service/                 # Python ML service
├── docs/                       # Documentation
└── docker-compose.yml          # Local development
```